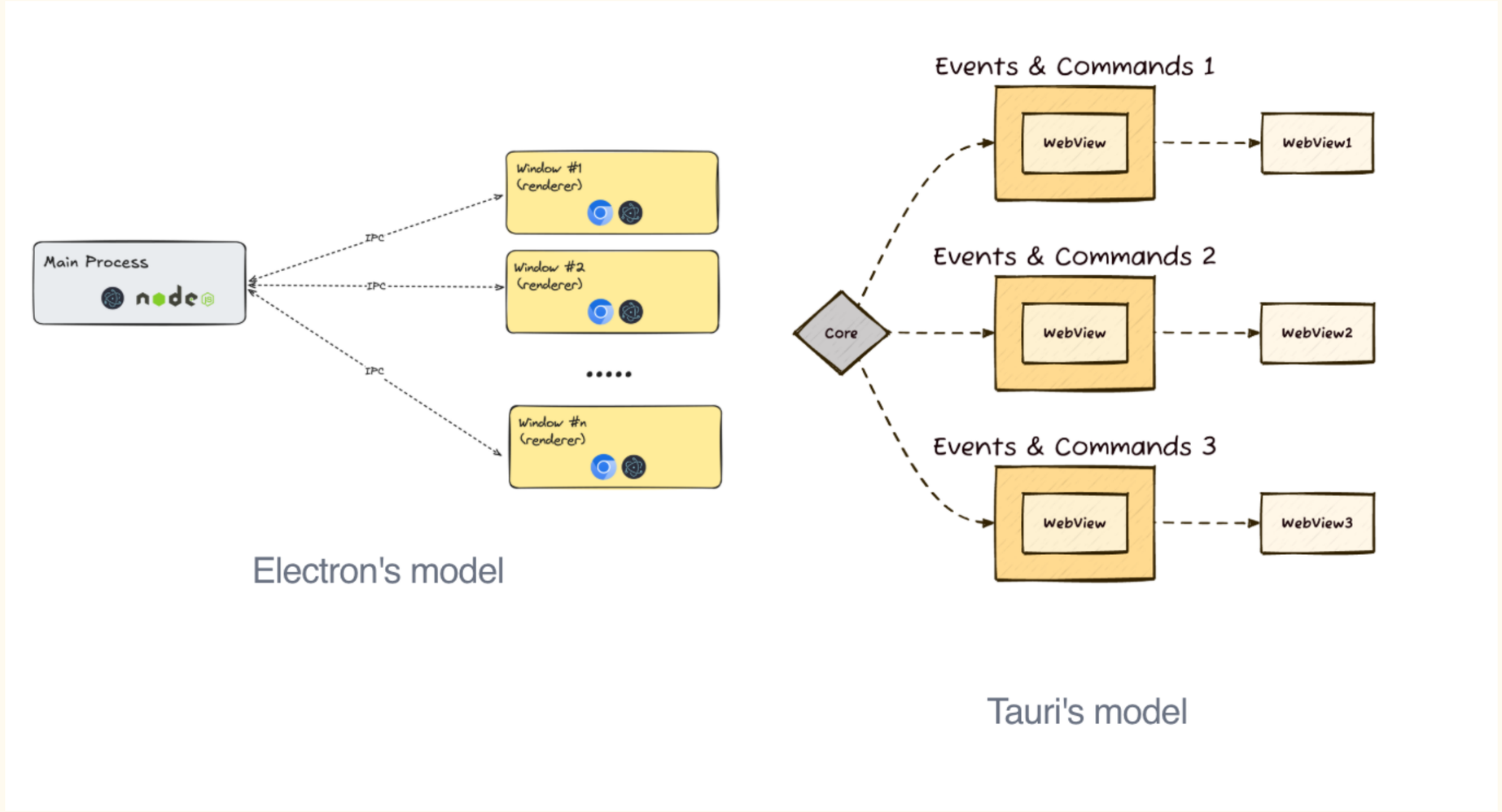
Tauri native apps

I. Lesson

- Created in 2019
- Tauri 2.0 in Feb. 2024

Electron vs Tauri



Comparison

Feature	Tauri 🦀	Electron ⚛️
Startup time	🏎️ Fast	🏎️ Fast
Memory Usage (Benchmark)	`172 MB`	`409 MB`
Browser technology	`WebView`	`Chromium`
Backend language	`Rust`	`JavaScript`
Build time (Initial)	🦕 Slow	🏎️ Fast
Bundle size (Benchmark)	`8.6 MiB`	`244 MiB`

Tauri WebView

Electron ships a complete Chromium engine inside every app, along with Node.js to access operating system features. This gives developers a lot of power and consistency across platforms, but it also means the app carries a heavy runtime wherever it goes.

Tauri takes a different path. It leans on the system's built-in WebView for rendering the interface, while keeping a Rust core for anything that needs deeper access to the operating system. By not bundling an entire browser, Tauri apps are leaner by default.

Tauri WebView process

Instead of bundling the entire Chromium engine, Tauri uses the **operating system's native WebView component** to render the UI. This component generally weighs less than a full browser engine. Currently, Tauri uses:

- WebView2 on Windows (based on Microsoft Edge/Chromium)
- WKWebView on macOS (based on Safari/WebKit)
- WebKitGTK on Linux

By relying on the system's WebView, **Tauri creates smaller app bundles, but this comes at a cost.**

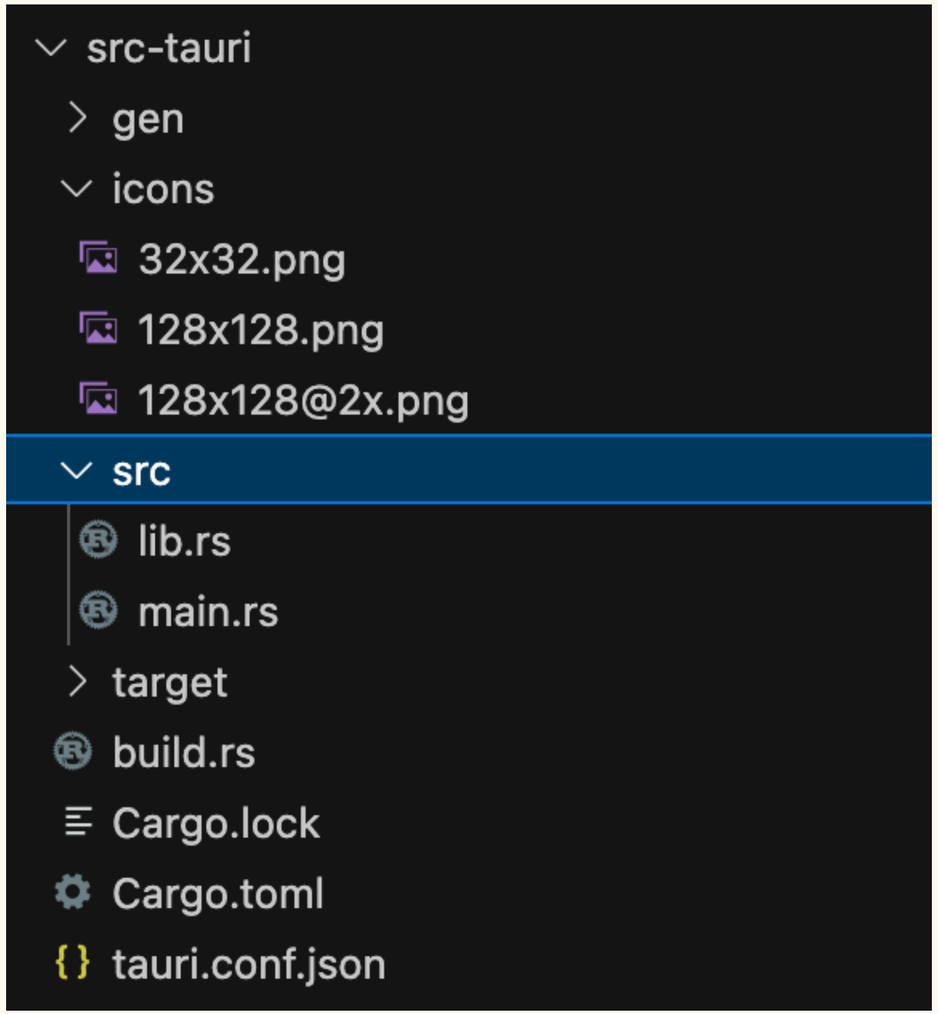
Plugins

In Tauri, the same behavior is defined in a single Rust builder. Capabilities are added through plugins and commands, making it more concise:

```
// tauri/src/main.rs
tauri::Builder::default()
.plugin(tauri_plugin_updater::Builder::new().build())
.plugin(tauri_plugin_single_instance::init(|app, _args, _cwd| {
if let Some(window) = app.get_webview_window("main") {
let _ = window.set_focus();
let _ = window.show();
let _ = window.unminimize();
}
}))
.invoke_handler(tauri::generate_handler![
commands::get_system_info,
commands::restart_service,
])
.run(tauri::generate_context!())?;
```

II. Project

Architecture of src-tauri



Src-tauri —> Cargo.toml

```toml
[package]
name = "rustify_bootcamp_tauri"
version = "0.1.0"
description = "A native app wrapper for Rustify Bootcamp"
edition = "2024"

[workspace]

[features]
default = []
tauri = []

[lib]
name = "rustify_bootcamp_tauri"
crate-type = ["staticlib", "cdylib", "rlib"]

[build-dependencies]
tauri-build = { version = "2", features = [] }

[dependencies]
tauri = { version = "2", features = ["devtools"] }
tauri-plugin-opener = "2"
```

Src-tauri —> main.rs

```rust
// Prevents additional console window on Windows in release, DO NOT REMOVE!!
#![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]

fn main() {
    rustify_bootcamp_tauri::run()
}
```

Src-tauri —> lib.rs

```rust
pub fn run() {
    tauri::Builder::default()
        .plugin(tauri_plugin_opener::init())
        .setup(|app| {
            let mut builder = tauri::WebviewWindowBuilder::new(
                app,
                "main",
                tauri::WebviewUrl::External("http://localhost:3000".parse().unwrap()),
            );

            builder.build()?;
            Ok(())
        })
        .run(tauri::generate_context!())
        .expect("error while running tauri application");
}
```

Src-tauri —> tauri.conf.json

```
{
  "$schema": "https://schema.tauri.app/config/2",
  "productName": "Rustify W4 D20",
  "version": "0.1.0",
  "identifier": "rs.rustify",
  "build": {
    "beforeDevCommand": "cargo leptos watch",
    "devUrl": "http://localhost:3000",
    "beforeBuildCommand": "cargo leptos build --release"
  },
  "app": {
    "withGlobalTauri": true,
    "security": {
      "csp": null
    }
  },
  "bundle": {
    "active": true,
    "targets": "all",
    "icon": [
      "icons/32x32.png",
      "icons/128x128.png",
      "icons/128x128@2x.png"
    ],
    "resources": []
  }
}
```

Server functions

```rust
/// * We need to specify input & output for mobile targets (Android, iOS) to work.
#[server(GetPosts, "/api", input = GetUrl, output = Json)]
pub async fn get_posts() -> Result<Vec<PostDto>, ServerFnError> {
    let resp = match reqwest::get(format!("{BASE_URL}?_limit={LIMIT}")).await {
        Ok(response) => response,
        Err(err) => return Err(ServerFnError::ServerError(err.to_string())),
    };

    let posts = match resp.json::<Vec<PostDto>>().await {
        Ok(data) => data,
        Err(err) => return Err(ServerFnError::ServerError(err.to_string())),
    };

    Ok(posts)
}
```

Let’s code!

Your turn!